

API SECURITY RISK ANALYSIS REPORT

API Tested: JSONPlaceholder

URL: <https://jsonplaceholder.typicode.com>

Prepared by: NORAH CYPRIAN NGILIULE

CIN ID: FIT/MAY26/CS8136

Date: 21st May 2026

For: Future Interns Task 3

EXECUTIVE SUMMARY

This assessment involved testing the JSONPlaceholder API using multiple controlled requests to evaluate its security posture.

The analysis identified several security weaknesses within the API, including lack of authentication and insufficient access controls.

Key Security Issues Identified:

- Unauthorized access to user data due to absence of authentication
- IDOR vulnerability allowing access to different user records by modifying the user ID
- Missing HTTP security headers that help protect against common web attacks
- No rate limiting, allowing unlimited number of requests

Overall Risk Level: **MEDIUM**

The API requires implementation of authentication mechanisms, proper access control, security headers, and rate limiting to improve its security posture and reduce exposure to potential attacks.

API Endpoints Tested

1. GET /users - Get all users
2. GET /users/1 - Get user 1
3. GET /users/2 - Get user 2 (IDOR test)
4. POST /posts - Create a fake post
5. Headers check - Look for security headers

FINDING 1: No login required

Problem: Anyone can access user data without a username or password.

What I did

I used Postman to send a GET request to the API endpoint:

GET <https://jsonplaceholder.typicode.com/users>

What happened: The API returned everyone's name, email, address, and phone number.

```
1 [{"id": 1,
2   "name": "Leanne Graham",
3   "username": "Bret",
4   "email": "Sincere@april.biz",
5   "address": {
6     "street": "Kulas Light",
7     "suite": "Apt. 556",
8     "city": "Gwenborough",
9     "zipcode": "92998-3874",
10    "geo": {
11      "lat": "-37.3159",
12      "lng": "81.1496"
13    }
14  },
15  "phone": "1-770-736-8031 x56442",
16  "website": "hildegard.org",
17  "company": {
18    "name": "Romaguera-Crona",
19    "catchPhrase": "Multi-layered client-server neural-net",
20    "bs": "harness real-time e-markets"
21  }
22 },
23 {
24   "id": 2,
25   "name": "Ervin Howell",
26   "username": "Antonette",
27   "email": "Shanna@melissa.tv",
28   "address": {
29     "street": "Victor Plains",
30     "suite": "Suite 879",
31     "city": "Wisokyburgh",
32     "zipcode": "90566-7771",
33     "geo": {
```

Why this is bad

- Hackers can steal all user data
- Spammers can collect email addresses
- Attackers can learn who uses the system

Risk Level: **MEDIUM**

How to fix: Require an API key or login token for every request

FINDING 2: IDOR VULNERABILITY

Problem: Sequential user IDs allow unauthorized access to multiple user records by changing the ID in the endpoint.

What I did

I used Postman to send multiple GET requests to different API endpoints:

GET <https://jsonplaceholder.typicode.com/users/1>

GET <https://jsonplaceholder.typicode.com/users/2>

User with id 1

```
1  {
2    "id": 1,
3    "name": "Leanne Graham",
4    "username": "Bret",
5    "email": "Sincere@april.biz",
6    "address": {
7      "street": "Kulas Light",
8      "suite": "Apt. 556",
9      "city": "Gwenborough",
10     "zipcode": "92998-3874",
11     "geo": {
12       "lat": "-37.3159",
13       "lng": "81.1496"
14     }
15   },
16   "phone": "1-770-736-8031 x56442",
17   "website": "hildegard.org",
18   "company": {
19     "name": "Romaguera-Crona",
20     "catchPhrase": "Multi-layered client-server neural-net",
21     "bs": "harness real-time e-markets"
22   }
23 }
```

User with id 2

```
{
  "id": 2,
  "name": "Ervin Howell",
  "username": "Antonette",
  "email": "Shanna@melissa.tv",
  "address": {
    "street": "Victor Plains",
    "suite": "Suite 879",
    "city": "Wisokyburgh",
    "zipcode": "90566-7771",
    "geo": {
      "lat": "-43.9509",
      "lng": "-34.4618"
    }
  },
  "phone": "010-692-6593 x09125",
  "website": "anastasia.net",
  "company": {
    "name": "Deckow-Crist",
    "catchPhrase": "Proactive didactic contingency",
    "bs": "synergize scalable supply-chains"
  }
}
```

Why this is bad

- Hackers can write a script to download every user.
- No permission checks between users.
- Private data becomes public.

Risk Level: **MEDIUM**

How to fix

- Use random IDs (UUIDs) instead of 1,2.

- Check if the user is allowed to see each record.
- Require authentication for all requests.

FINDING 3: Missing security headers

Problem: The API does not include important HTTP security headers that help protect applications from common web attacks.

What I did

I used Postman to inspect the response headers of the following endpoint
GET <https://jsonplaceholder.typicode.com/users>

Missing headers found:

- Content-Security-Policy (blocks malicious scripts)
- X-Frame-Options (prevents clickjacking)
- X-Content-Type-Options (prevents file type attacks)

```

1  {
2    "status": "HTTP/2 200",
3    "date": "Thu, 21 May 2026 08:46:45 GMT",
4    "content-type": "application/json; charset=utf-8",
5    "access-control-allow-credentials": "true",
6    "cache-control": "max-age=43200",
7    "etag": "W/\"160d-1eMSsxeJRfnVLRBmYJSbCiJZ1qQ\\\"",
8    "expires": "-1",
9    "nel": "{\\\"report_to\\\":\\\"heroku-nel\\\",\\\"response_headers\\\":[\\\"Via\\\"],\\\"max_age\\\":3600,\\\"success_fraction\\\":0.01,\\\"failure_fraction\\\":0.1}\"",
10   "report-to": "{\\\"group\\\":\\\"heroku-nel\\\",\\\"endpoints\\\":[{\\\"url\\\":\\\"https://nel.heroku.com/reports?s=PB3OQYtmpF%2FWUssyAhG8UVS7zHD7uZWe3KKz7\\\"}],\\\"max_age\\\":604800}\"",
11   "reporting-endpoints": "heroku-nel=\\\"https://nel.heroku.com/reports?s=PB3OQYtmpF%2FWUssyAhG8UVS7zHD7uZWe3KKz7Dwfb70x3D&sid=e11707d5-02a7-4\\\"",
12   "server": "cloudflare",
13   "vary": "Origin, Accept-Encoding",
14   "via": "2.0 heroku-router",
15   "x-content-type-options": "nosniff",
16   "x-powered-by": "Express",
17   "x-ratelimit-limit": "1000",
18   "x-ratelimit-remaining": "997",
19   "x-ratelimit-reset": "1777075245",
20   "age": "18403",
21   "cf-cache-status": "HIT",
22   "cf-ray": "9fd997e15ecd8468-JNB",
23   "alt-svc": "h3=\\\"443\\\"; ma=86400"
24 }
25
26

```

Why this is bad

- Attackers could inject malicious code.
- Your website could be embedded in a fake page.
- Browsers might misinterpret file types.

Risk Level: **LOW**

How to fix

- Add these headers to API responses
- Content-Security-Policy: default-src 'self'
- X-Frame-Options: DENY
- X-Content-Type-Options: nosniff

FINDING 4: No rate limit

Problem: The API does not enforce any rate limiting, allowing unlimited requests to be made within a short period of time.

What I did

I used Postman to send multiple rapid requests to the API endpoint:

GET <https://jsonplaceholder.typicode.com/users>

I sent approximately 20 requests within 2 seconds and monitored the responses in Postman.

Result: All requests were successfully processed and returned HTTP 200 OK responses. No request was blocked or rate-limited.

Why this is bad

- The API is vulnerable to denial-of-service (DoS) attacks
- Attackers can perform brute-force attempts without restriction
- Automated bots can scrape large amounts of data quickly

Risk Level: **LOW**

How to fix

- Limit: 100 requests per minute per IP.
- Response when exceeded: HTTP 429 Too Many Requests .

RECOMMENDATIONS

1. Add Authentication

- Require API keys or login tokens for all requests
- Prevent anonymous access to sensitive user data
- Ensure only authorized users can access the API

2. Fix IDOR (Insecure Direct Object Reference)

- Replace sequential IDs (1, 2, 3) with random UUIDs
- Implement proper authorization checks before returning data
- Ensure users can only access permitted resources

3. Add Security Headers

- Implement key HTTP security headers such as:
 - Content-Security-Policy (CSP)
 - X-Frame-Options
 - X-Content-Type-Options
- These help protect against XSS, clickjacking, and MIME sniffing attacks

4. Add Rate Limiting

- Limit requests per IP address (e.g. 100 requests per minute)
- Return HTTP 429 Too Many Requests when limit is exceeded
- Protect against brute-force attacks, DoS attacks, and data scraping

CONCLUSION

The JSONPlaceholder API demonstrates several security weaknesses that would be unacceptable in a real world production environment. The most critical issues identified include the absence of authentication and the presence of IDOR vulnerability which allows unauthorized access to user data by simply modifying the user ID parameter. These weaknesses could potentially lead to full data exposure if implemented in a real business system. Additionally, missing security headers and lack of rate limiting further increase the API's exposure to common web-based attacks. Implementing authentication mechanisms, proper access controls, security headers, and rate limiting would significantly improve the overall security posture of the API.

Note: All testing was conducted in a controlled, read only manner, no exploitation, brute force, or malicious activity was performed during this assessment.

